Label Management Compiler

Course: Automata Theory and Compiler Design

Institution: Amrita Vishwa Vidyapeetham

Abstract

This report presents the design and implementation of a multi-phase compiler system focused on label management in C++ code. The system performs lexical analysis, syntax analysis, semantic analysis, duplicate detection, unused label identification, and reuse optimization. The compiler successfully detects duplicate labels, tracks usage patterns, identifies dead code, and provides intelligent reuse recommendations for unused labels. The system was validated against 12 comprehensive test cases, achieving a 100% pass rate.

1. Problem Statement

Title: Label Management Compiler with 6-Phase Pipeline

Objective:

Design and implement a multi-phase compiler system that analyzes C++ code with a focus on label management. The system must detect duplicate labels, track label usage, identify unused labels, and provide optimization suggestions for label reuse.

Core Requirements:

Requirement	Description
Avoid duplicate labels	Detect and report multiple definitions of the same label
Track label usage	Monitor which labels are referenced by goto statements
Identify unused labels	Detect labels that are defined but never used
Provide reuse optimization	Suggest recycling of unused labels for future use

Input:

C++ source code containing labels (e.g., start:, cleanup:) and goto statements.

Expected Output:

Unique label list with usage statistics, duplicate detection report, unused label identification, label reuse recommendations, parse tree representation, and Three-Address Code (TAC) optimization notes.

2. System Architecture

The compiler implements a 6-phase pipeline architecture:

Phase	Name	Function
1	Lexical Analysis	Converts source code into tokens
2	Syntax Analysis	Builds parse tree and verifies grammar
3	Semantic Analysis	Creates symbol table with usage tracking
4	Duplicate Detection	Identifies duplicate labels and functions
5	Unused Detection	Flags dead labels with zero references
6	Reuse Optimization	Generates TAC and reuse recommendations

3. Phase-Wise Implementation and Output

Phase 1: Lexical Analysis (Scanner)

The scanner reads source code character by character, identifying tokens including keywords, identifiers, numbers, operators, punctuation, and labels. Label definitions are identified by an identifier followed by a colon (:).

Sample Input:

```
int main() {
    int a = 5;
start:
    if (a > 0) goto start;
    return a;
}
```

Output – Token Stream:

Line	Lexeme	Token Type	
1	int	KEYWORD	
1	main	IDENTIFIER	
1	(	PUNCT	
1	)	PUNCT	
1	{	PUNCT	
2	int	KEYWORD	
2	a	IDENTIFIER	
2	=	PUNCT	
2	5	NUMBER	
2	;	PUNCT	
3	start	LABEL_DEF	
4	if	KEYWORD	
4	a	IDENTIFIER	
4	>	OP	

Line	Lexeme	Token Type	
4	0	NUMBER	
4	goto	KEYWORD	
4	start	IDENTIFIER	
4	;	PUNCT	
5	return	KEYWORD	
5	a	IDENTIFIER	
5	;	PUNCT	
6	}	PUNCT	

Total Tokens: 24 | Labels Found: 1 (start)

Phase 2: Syntax Analysis (Parser) + Parse Tree

The parser verifies syntactic correctness and constructs an Abstract Syntax Tree (AST). It handles operator precedence and expression evaluation.

Parse Tree Output:

```
TranslationUnit
  ├── Function: main
  ├── Label: start
  ├── Assignment
  │   ├── Identifier: a
  │   └── Literal: 5
  ├── If-Statement
  │   ├── Condition: >
  │   │   ├── Identifier: a
  │   │   └── Literal: 0
  │   └── Goto: start
  └── Return: a
```

Expression Tree Example (for x = a + b * 2):

```
        =
       / \
      x   +
         / \
        a   *
           / \
          b   2
```

Phase 3: Semantic Analysis (Symbol Table)

The semantic analyzer builds a symbol table tracking each label's definition status and usage count.

Label Symbol Table Output:

Label	Defined	Uses	Status
start	Yes	2	Used

Label	Defined	Uses	Status
cleanup	Yes	0	Unused

Usage Statistics:

Total Labels Defined: 2

Labels Used: 1 (50%)

Labels Unused: 1 (50%)

Phase 4: Duplicate Detection

This phase identifies duplicate label and function definitions by scanning the symbol table for multiple definitions of the same name.

Duplicate Detection Report:

```
DUPLICATE DETECTION REPORT
==================================

Duplicate Labels Found:
  'duplicate' - defined at lines 3 and 5

Duplicate Functions Found:
  'process' - defined at lines 1 and 4

Summary: Total Duplicates: 2
```

When no duplicates exist:

```
  No duplicate labels or functions detected.
```

Phase 5: Unused Label Detection (Dead Code Analysis)

Identifies labels that are defined but never referenced by any goto statement.

Unused Label Report:

```
UNUSED LABEL DETECTION
==================================

Unused Labels (Dead Code):
  cleanup (line 8) - defined but never jumped to
  exit (line 12) - defined but never referenced

Summary:
  Total Labels Defined: 3
  Labels Used: 1 (33%)
  Unused Labels: 2 (67%)

Recommendation: Remove or reuse unused labels.
```

Phase 6: Reuse Optimization + Three-Address Code (TAC)

The final phase generates reuse recommendations and Three-Address Code. Unused labels are placed in a reuse pool for future allocation.

Reuse Optimization Report:

```
LABEL REUSE OPTIMIZATION
================================

Unused Label Pool:
  cleanup (line 8) - eligible for reuse
  exit (line 12) - eligible for reuse

Reuse Suggestions:
  1. Label 'cleanup' can be repurposed for error handling
  2. Label 'exit' can be used as alternative termination point
```

Three-Address Code (TAC):

```
main:
    a = 5
    b = 3
    x = a + (b * 2)
    y = (a - b) * (a + b)

start:
    if x > y goto cleanup
    x = x + 1
    goto start

cleanup:
    return x
```

No Waste Scenario:

```
  No wasted labels — perfect efficiency.
  All labels are actively used. Zero-waste recommendation.
```

4. Test Suite Results

The compiler was validated against 12 comprehensive test cases covering label usage, duplicate detection, unused label identification, and reuse optimization.

#	Test Case	Description	Unused	Dupes	Status
1	Basic labels + usage	Simple label with goto	0	None	PASS
2	Unused label detection	Label defined but never used	1	None	PASS
3	Duplicate label	Same label defined twice	0	1	PASS
4	Mixed used/unused	Multiple labels with varied usage	2	None	PASS
5	Duplicate function	Same function defined twice	0	1	PASS

#	Test Case	Description	Unused	Dupes	Status
6	Reuse candidate pool	Multiple unused labels for recycling	2	None	PASS
7	All labels used	All labels referenced	0	None	PASS
8	Expression with labels	Labels in expression context	0	None	PASS
9	Multiple duplicates + unused	Combined duplicate and unused	1	1	PASS
10	No labels	Code without any labels	0	None	PASS
11	Advanced reuse suggestion	Unused label with reuse potential	1	None	PASS
12	Function + label coexistence	Both functions and labels present	1	None	PASS

Overall Pass Rate: 12/12 (100%)

5. Innovation: Unused Label Reuse Strategy

The system implements an innovative Label Recycling Strategy that distinguishes it from traditional compilers:

Detection Phase: Identifies all labels with zero goto references
Pool Management: Maintains a pool of reusable label names
Reuse Recommendation: Suggests repurposing unused labels for new control flow structures
TAC Optimization: Generates optimized three-address code that incorporates recycled labels

Benefits:

- Reduces symbol table overhead
- Improves code maintainability
- Enables better memory utilization
- Facilitates dead code elimination
- Provides actionable optimization suggestions

6. Conclusion

The C++ Label Compiler successfully implements a complete 6-phase compilation pipeline with a focus on label management. The system effectively detects duplicate labels, tracks usage patterns, identifies dead labels, and provides intelligent reuse recommendations. The 12-test validation suite confirms the compiler's correctness and robustness. The innovative reuse optimization phase demonstrates practical application of compiler theory to code quality improvement.